Who’s that bot? A Pokemon Showdown Bot Through Reinforcement Learning

Abstract

...

Related Works

Winning at Pokémon Random Battles Using Reinforcement Learning

Wang (2024) presents a bot designed to play random battles in Generation 4. The agent is modeled as a Partially Observable Markov Decision Process (POMDP), trained using Proximal Policy Optimization (PPO), and further improved at inference time through Monte Carlo Tree Search (MCTS).

Introduction

Problem Statement

Using heuristics to play games like Pokémon has historically been an effective approach. However, it raises the question of whether playing exclusively with heuristics is truly optimal. Pokémon Showdown provides a bot that plays using simple heuristics, so we created an agent with three available strategies: playing with simple heuristics, attacking with the highest base power move, and choosing a random move. This allows us to evaluate whether playing outside of pure heuristics can be beneficial.

Background

Pokemon VGC

Pokémon VGC is Nintendo's official format for Pokémon battles. It consists of 2-on-2 matches, where each player brings a team of 6 Pokémon, but at the start of each battle they select only 4 of them, and of those 4 only 2 can be on the field at the same time. Each Pokémon can hold an equipped item, and throughout the battle players take turns simultaneously choosing an action for each of their two active Pokémon, either attacking or switching in another member of the team. The winner is whoever leaves the opponent with no Pokémon remaining, or whoever has more Pokémon standing when the battle timer runs out.

Pokemon Showdown

Pokémon Showdown is a free, open-source Pokémon battle simulator that runs directly in the browser. Unlike the official video games, where players need to invest hundreds of hours obtaining and training their Pokémon, Showdown allows anyone to build a competitive team in a matter of minutes and start battling right away. This makes it the perfect tool for the competitive community to practice, test strategies, and prepare for official tournaments.And in this case train our IA.

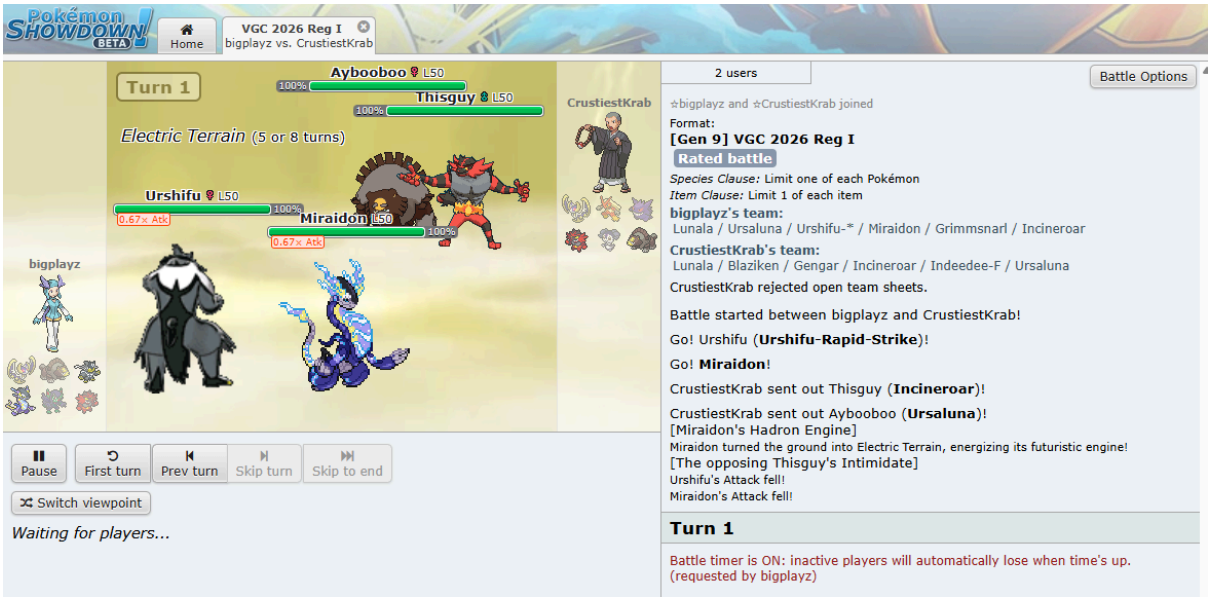


Figure 1.1: A Gen 9 VGC 2026 regulation I match in Pokemon Showdown.

Environment

Environment Description

The agent will be trained in a local Pokémon Showdown server running on Node.js, interacting with it through the Python library poke-env. Battles will take place in the VGC 2026 Regulation M-A, and the agent will use a fixed team built by a human, which will remain the same throughout the entire training process. A battle ends when all Pokémon on one side have fainted.

State Space

At the start of each turn, the agent can receives a complete state of the battle, including all the information of the Pokémons. Talking about the battle it ican ncludes the current weather and terrain, any field effects such as Tailwind or Trick Room, which Pokémon on either side have fainted, and the current turn number. For this agent the scope of the stace space wil be reduce.

State Space:
Own side**:**

- Current HP of all its active Pokémon (2 numbers)
- Moves available (8 moves)

Opponent's side:

- Current HP of all opponent's active Pokémon (2 numbers)

Action Space

Each turn, the agent must choose the strategy it will follow. The available strategies are a random move, the highest base power move, or simple heuristics. Once the agent selects a strategy, the corresponding Pokémon Showdown algorithm executes the action for both active Pokémon.

Action Space:

- Random Move
- Max Base Power
- Simple Heuristics

Reward Function

The reward function is based on Wang (2024). The agent receives a reward of +1 upon winning a battle, -1 upon losing, and 0 on every other turn. This reward structure allows the agent to learn strategies that play out over multiple turns, such as moves with effects that last several turns or setups that only pay off later in the battle, something that would not be possible if the reward function assigned value to individual turn outcomes.

Methods

Training

The agent will play against itself to train the model. We will save the SimpleHeuristicsPlayer, a pokemon bot, for validation. According to Wang (2024), the bot is easy enough for the agent to beat, but difficult enough for it to learn from. This balance is precisely why RandomPlayer and MaxBasePowerPlayer were not used to train the model, RandomPlayer selects actions at random, meaning the agent would learn nothing meaningful from it, while the MaxBasePowerPlayer always picks the highest base power move, which could teach the agent the bad habit of always assuming that the strongest move is the best choice.

Validation

The metrics used to measure the model's success will be based on Wang (2024). We will use win rate, against SimpleHeuristicsPlayer, as the primary success metric, measured every 200 battles, which according to the paper corresponds to approximately 20,000 steps against the bot.

Agent 1

Architecture

The agent uses a deep neural network with 12 inputs, followed by 3 fully connected layers of 256, 128, and 64 neurons respectively, with ReLU activation, ending in an output layer with linear activation. The output produces 3 Q-values, one for each strategy the agent can choose(A random move, The strongest attack or A simple heuristic move). The network uses the Adam optimizer with a learning rate of 0.001 and mean squared error as the loss function. Softmax is not used because we care about Q-values and not the probability of each action, and mean squared error is used instead of cross-entropy because the latter is for classification, while mean squared error better penalizes large errors during training.

Training

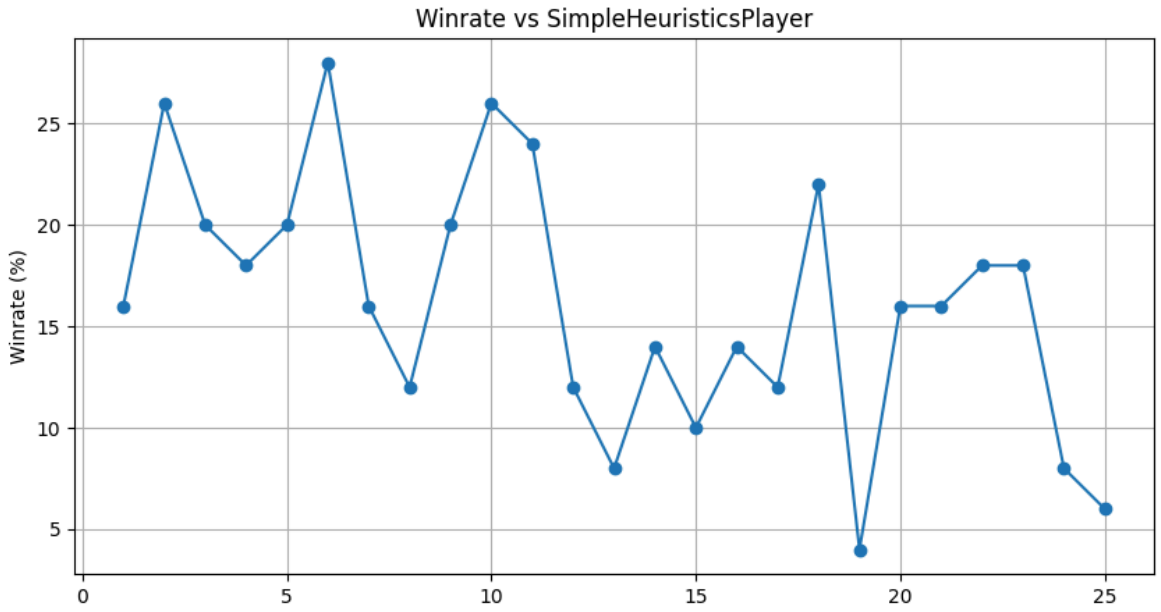
At each turn, the agent receives the current battle state: the health points of the 4 active Pokémon and a binary value (0 or 1) indicating the availability of each Pokémon's moves. With this state, the agent calls the model to return an action.

Starting from the second turn, the agent stores in memory the previous battle state, the previous action taken, the reward received, the current state and the current action. The agent requires a minimum of 64 stored experiences before it can begin learning.
During training, the agent samples 64 random experiences from memory and uses the Bellman equation to calculate the target Q-value for each experience. This target Q-value is then used to adjust the network's parameters via backpropagation.

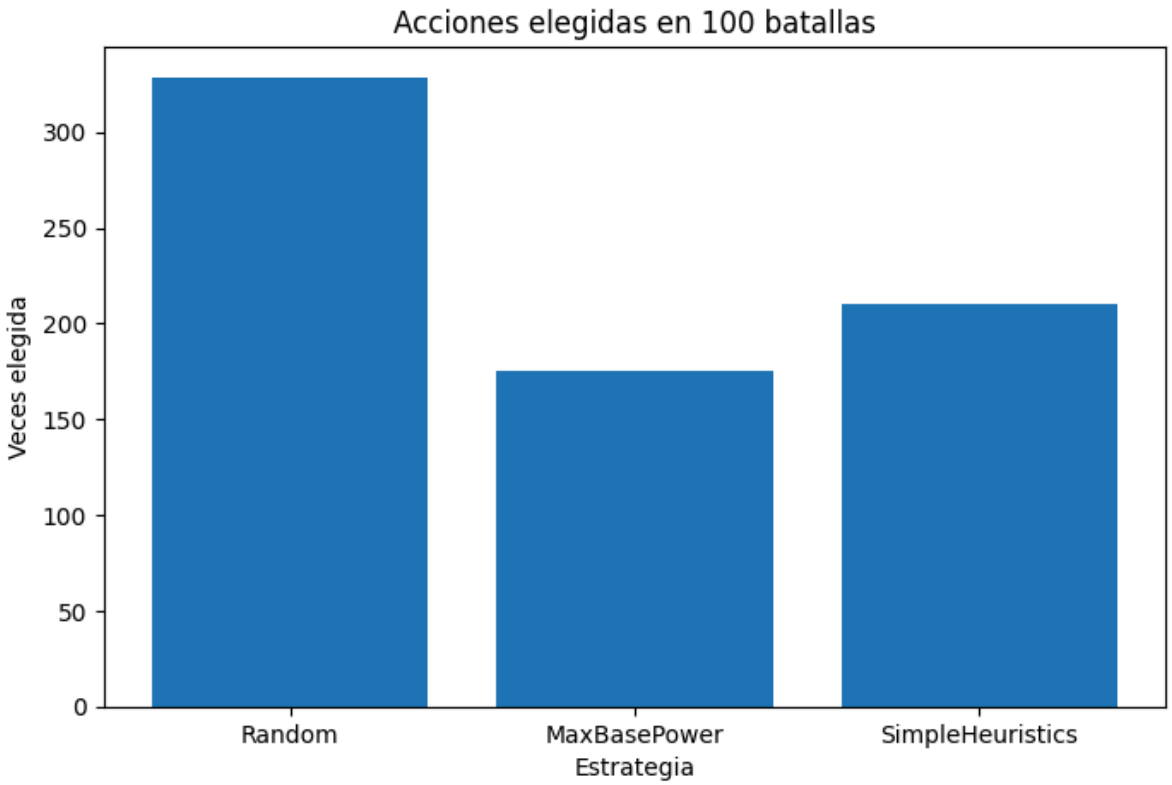
When choosing an action, the agent uses an epsilon-greedy policy. Epsilon starts at 1.0 and decays over time. At each turn, a random value between 0 and 1 is generated, if it is less than epsilon, the agent explores by choosing a random action; otherwise, its action is based on what it has learned by choosing the action with the highest Q-value.

$$Q(s_t, a_t) = r_t + \boxed{\gamma Q(s_{t+1}, a_{t+1})}$$

Results



The graph shows the agent's win rate against SimpleHeuristicsPlayer over 25 validation checkpoints, each taken after 200 training battles, for a total of 5,000 self-play battles. Training took approximately 6 hours. The win rate oscillates between 4% and 28% throughout training with no clear upward trend, and ends near 6% at the final checkpoint.



This graph shows that the agent chose random moves most of the time, when in fact that strategy should only be used occasionally to avoid being predictable. The agent's best option would have been simple heuristics, while max base power should be its second choice when an aggressive play is needed. Perhaps too few battles were played for the agent to learn that.

Conclusions and future steps

In conclusion, this model does not appear to be learning to make optimal decisions. Currently the agent can only choose between 3 strategies: random move, max base power move, and simple heuristics move. One would expect the agent to learn to choose the simple heuristics strategy most of the time, which would yield at least a 50% win rate.

Several factors may explain this failure. First, epsilon decayed too quickly, by the time the first validation checkpoint was reached, epsilon had already dropped to its minimum value, meaning the agent stopped exploring before learning anything meaningful. For the next attempt, epsilon will decay once per battle instead of once per turn.

Second, the agent learns from 64 random experiences sampled from its entire history. In Pokémon, early-game decisions have a greater impact on the outcome than late-game ones, so sampling randomly may not be the best idea for learning. A better approach would be to learn from the last 5 to 10 turns instead, and increasing gamma from 0.99 to 1.0 could also help the model better value long-term outcomes.

Third, the action space may be too limited. Initially the idea was to allow the agent to choose specific moves, but due to implementation difficulties and training tim, it was simplified to only 3 strategies. It now appears that significantly more training time will be needed, and allowing the agent to choose moves more specifically may be necessary to achieve competitive performance.

Finally, we will increase the state vector, so the agent can use more knowledge of the environment to make its predictions.

Agent 2

Architecture

The architecture of this model is identical to the previous one, with the same number of neurons, layers, activation functions, inputs, and outputs. The only changes made are to the training process and how often the model parameters are updated.

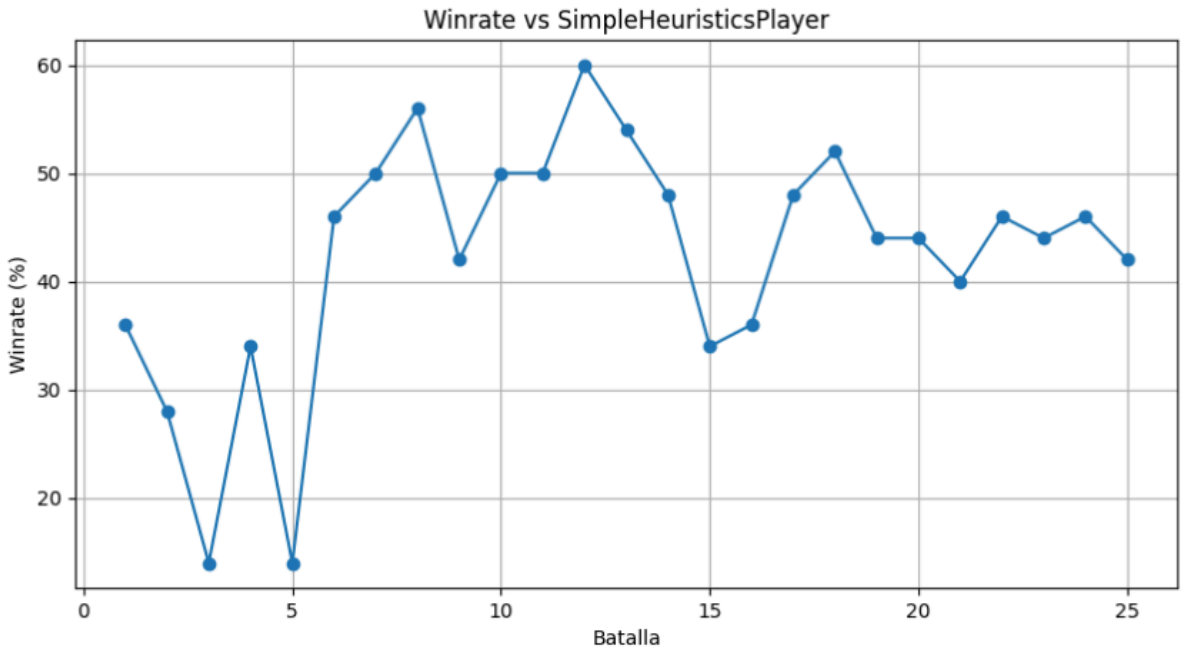
Training

The first change was to slow down the epsilon decay rate. According to Perkins et al. (2025) [3], a decay rate that is too passive can increase the temporal complexity of the algorithm yet a very aggressive decay can prevent it from finding the optimal policy. For this reason, epsilon decay was reduced from 0.995 to 0.9975, allowing the agent to explore more before converging.

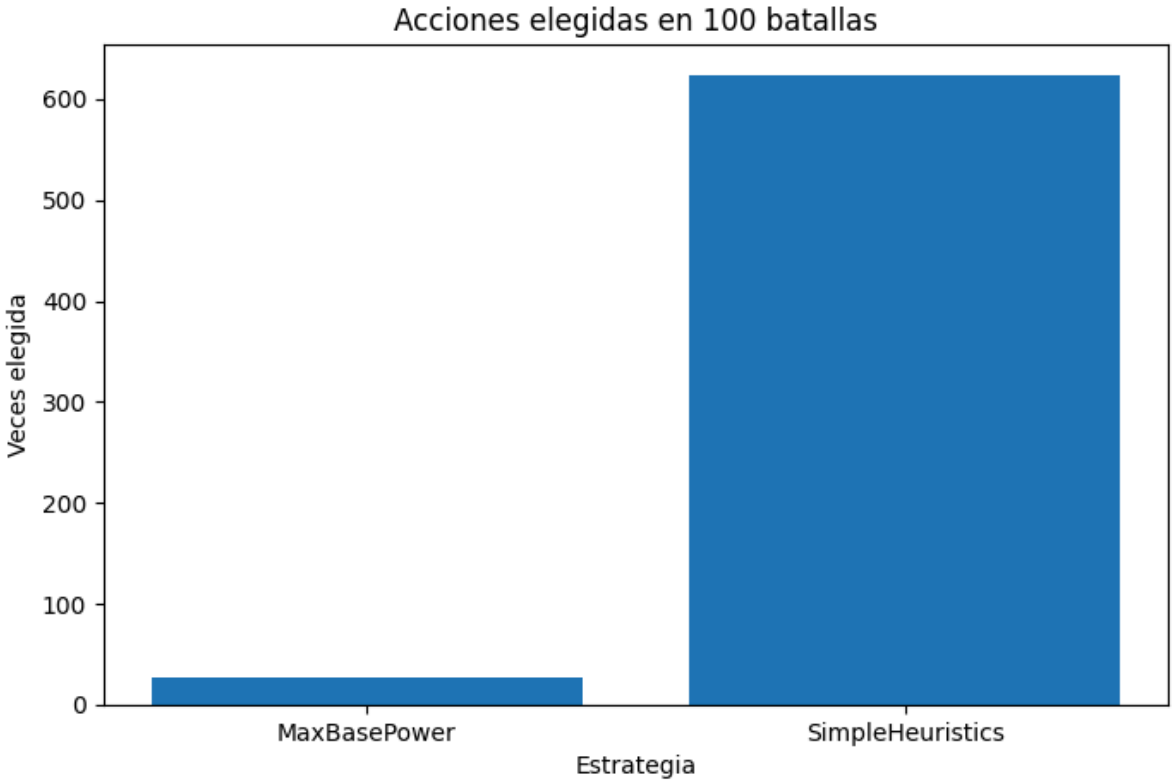
The second change was to increase gamma. According to Mnih et al. (2013) in Playing Atari with Deep Reinforcement Learning [2], future rewards are discounted by a factor of gamma for each step taken. This means that strategies that require more time to see their benefits need a higher gamma so that the reward signal is not lost. Based on Wang (2024), gamma was increased from 0.99 to 0.9999.

The third change, motivated by Fedus et al. (2020) in Revisiting Fundamentals of Experience Replay [4], addresses the replay buffer size. The abstract discusses how buffer size can be beneficial for some algorithms but not others, and highlights that controlling the replay ratio is important in RL algorithms. For this reason, instead of training on a fixed batch of 64 random experiences per turn, the model now learns from all the turns of the current battle at the end of each game, and replacing per-turn updates with per-battle updates.

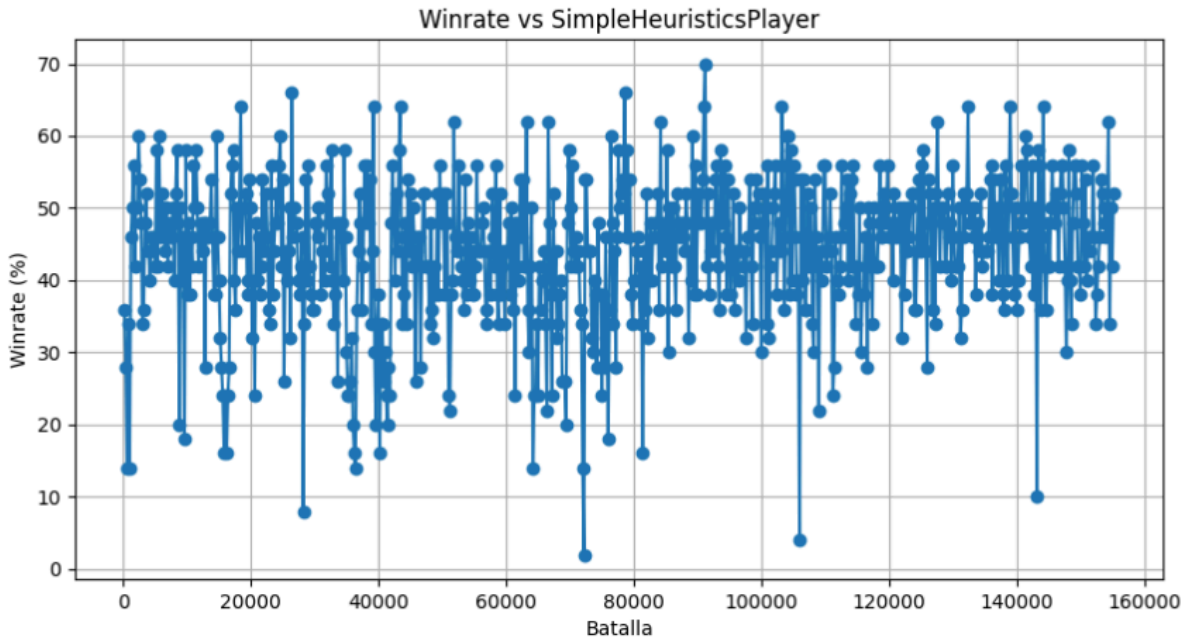
Results



The graph shows that at the beginning the agent tended to win less than 20% of the time, but after validation checkpoint 5 it improved and maintained a win rate between 35% and 60%. These metrics were obtained under the same conditions as the previous model: 5,000 training battles with validation against SimpleHeuristicsPlayer every 200 battles. The agent finished its last validation checkpoint with a 42% win rate.



Unlike the previous model, this agent learned to prefer simple heuristics as its primary strategy over random moves. The agent determined that to maximize victories, random moves should never be used. A low frequency of random moves was expected, similar to max base power, but a frequency of zero was not anticipated.



The model was left running overnight to observe the distribution of its win rates across a larger sample. Using the scipy library, out of 38,800 validation battles the agent won 17,075, yielding an average win rate of 44%. A one-tailed binomial test was performed to determine whether the agent wins more than 50% of the time at a 95% confidence level. The resulting

p-value of 0.997 indicates there is no statistical evidence that this agent wins more than 50% of battles against SimpleHeuristicsPlayer. The spikes observed in both directions can be attributed to the non-deterministic nature of Pokémon battles, where the same action does not always lead to the same outcome. Random factors such as move accuracy and damage variance are likely responsible for these fluctuations.

Results

Analysis and comparison

Model

Parameter	Agent 1	Agent 2
Epsilon decay	0.995 (per turn)	0.9975 (per battle)
Gamma	0.99	0.9999
Batch size	64 random experiences	All turns of current battle
Update frequency	Every turn	End of each battle

Results

Metric	Agent 1	Agent 2
Training battles	5,000	5,000
Last win rate (validation)	6%	42%
Win rate range	4% – 28%	35% – 60%
Preferred strategy	Random move	Simple heuristics

Conclusions

References

[1] Wang, J. (2024). Winning at Pokémon random battles using reinforcement learning (Master's thesis). Massachusetts Institute of Technology.

[2] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). *Playing Atari with deep reinforcement learning*. arXiv:1312.5602.

[3] Perkins, D., [et al.]. (2025). *DQN performance with epsilon greedy policies and prioritized experience replay*. arXiv:2511.03670.

[4] Fedus, W., Ramachandran, P., Agarwal, R., Bengio, Y., Larochelle, H., Rowland, M., & Dabney, W. (2020). *Revisiting fundamentals of experience replay*. arXiv:2007.06700.